ASP.NET Web API

Tag 2 – Konzepte in .NET und Datenmodelle

4-Tage-Kurs | Alexander Fischer, MSc

Wiederholung: Was wisst ihr noch?

Wiederholung: Was wisst ihr noch?

1. Welche HTTP-Methode ist nicht idempotent?

Wiederholung: Was wisst ihr noch?

1. Welche HTTP-Methode ist nicht idempotent?
2. Was ist der Unterschied zwischen 401 und 403?

Wiederholung: Was wisst ihr noch?

1. Welche HTTP-Methode ist nicht idempotent?
2. Was ist der Unterschied zwischen 401 und 403?
3. Was macht `[ApiController]`?

Wiederholung: Was wisst ihr noch?

1. Welche HTTP-Methode ist nicht idempotent?
2. Was ist der Unterschied zwischen 401 und 403?
3. Was macht `[ApiController]`?
4. Warum war die Liste in der Challenge `static`?

Wiederholung: Was wisst ihr noch?

1. Welche HTTP-Methode ist nicht idempotent?
2. Was ist der Unterschied zwischen 401 und 403?
3. Was macht `[ApiController]`?
4. Warum war die Liste in der Challenge `static`?
5. Was gibt `CreatedAtAction` zurück?

Services und Dependency Injection

Das Problem: feste Abhängigkeiten

```
public class GreetingController : ControllerBase
{
    private readonly GreetingService _service = new GreetingService();

    [HttpGet]
    public string Get() => _service.Greet("Kurs");
}
```

Das Problem: feste Abhängigkeiten

```
public class GreetingController : ControllerBase
{
    private readonly GreetingService _service = new GreetingService();

    [HttpGet]
    public string Get() => _service.Greet("Kurs");
}
```

- **Testbarkeit** – kein Austausch durch einen Fake möglich

Das Problem: feste Abhängigkeiten

```
public class GreetingController : ControllerBase
{
    private readonly GreetingService _service = new GreetingService();

    [HttpGet]
    public string Get() => _service.Greet("Kurs");
}
```

- **Testbarkeit** – kein Austausch durch einen Fake möglich
- **Austauschbarkeit** – neue Implementierung = jeden Controller ändern

Das Problem: feste Abhängigkeiten

```
public class GreetingController : ControllerBase
{
    private readonly GreetingService _service = new GreetingService();

    [HttpGet]
    public string Get() ⇒ _service.Greet("Kurs");
}
```

- **Testbarkeit** – kein Austausch durch einen Fake möglich
- **Austauschbarkeit** – neue Implementierung = jeden Controller ändern
- **Lebensdauer** – jede Instanz hat ihre eigenen Daten

Die Lösung: Dependency Injection

```
public class GreetingController : ControllerBase
{
    private readonly IGreetingService _service;

    public GreetingController(IGreetingService service)
        => _service = service;
}
```

- Der Controller sagt nur, **was** er braucht – ein Interface
- Der **DI-Container** entscheidet, **welche** Implementierung er bekommt
- Prinzip: **Inversion of Control** · in ASP.NET eingebaut

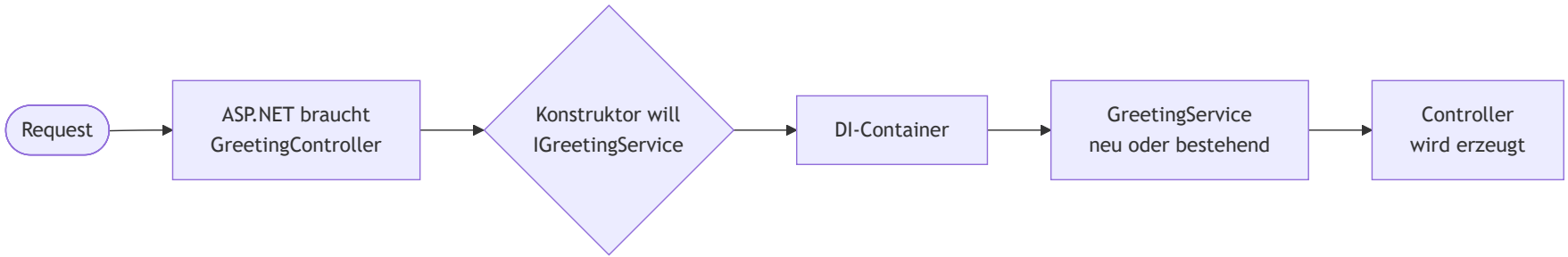
DI in drei Schritten

```
// 1. Interface
public interface IGreetingService { string Greet(string name); }

// 2. Implementierung
public class GreetingService : IGreetingService
{
    public string Greet(string name) => $"Hallo {name}";
}

// 3. Registrierung in Program.cs
builder.Services.AddScoped<IGreetingService, GreetingService>();
```

Wie der Container arbeitet



Rekursiv: Braucht der Service selbst etwas, wird das ebenfalls aufgelöst.

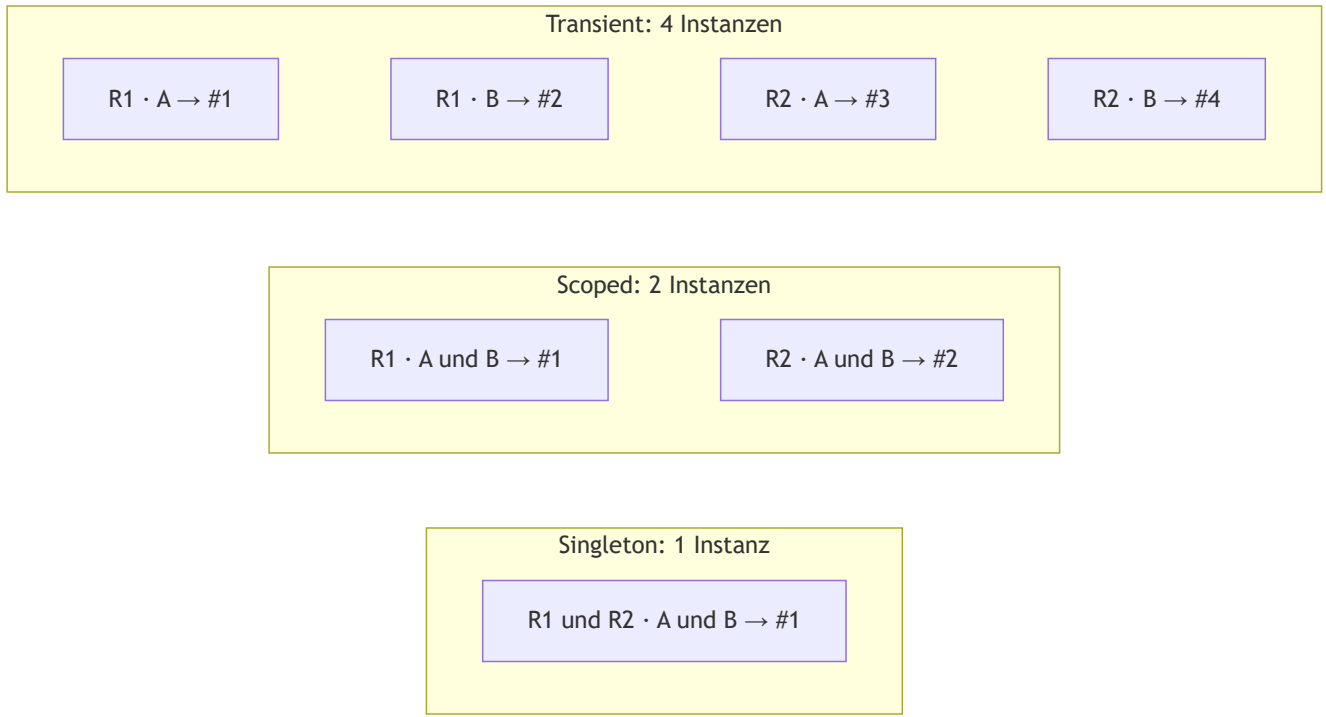
Lifetimes

Lifetime	Instanz	Typisch für
`AddTransient`	bei jeder Anforderung neu	leichte, zustandslose Dienste
`AddScoped`	eine pro HTTP-Anfrage	DbContext, Datenzugriff
`AddSingleton`	eine für die ganze Laufzeit	Caches, In-Memory-Daten

> Singletons werden von vielen Anfragen gleichzeitig genutzt – sie müssen **threadsicher** sein.

Lifetimes im Zeitverlauf

Zwei Requests, in jedem fordern zwei Stellen (A, B) den Service an:



> Ein Singleton darf keinen Scoped-Service nutzen – **Captive Dependency**.

Übung 5 – Service und Lifetimes

15 Minuten

1. `IGreetingService` mit `Greet(name)` und `Guid InstanceId`
2. `InstanceId` beim Erzeugen mit `Guid.NewGuid()` setzen
3. Service **zweimal** in einen `LifetimeController` injizieren
4. Beide Ids zurückgeben, Endpoint mehrmals aufrufen
5. `AddTransient`, `AddScoped`, `AddSingleton` nacheinander testen

Vorher notieren: Was erwartet ihr bei jeder Lifetime?

Lösung Übung 5

```
public interface IGreetingService
{
    Guid InstanceId { get; }
    string Greet(string name);
}

public class GreetingService
    : IGreetingService
{
    public Guid InstanceId { get; }
        = Guid.NewGuid();
    public string Greet(string name)
        => $"Hallo {name}";
}
```

```
public LifetimeController(
    IGreetingService a,
    IGreetingService b)
{
    _a = a;
    _b = b;
}

[HttpGet]
public object Get() => new {
    A = _a.InstanceId,
    B = _b.InstanceId };
```

Ergebnis Übung 5

Lifetime	A und B im selben Request	Nächster Request
Transient	verschieden	wieder neue Ids
Scoped	gleich	neue Id
Singleton	gleich	dieselbe Id

> Welche Lifetime braucht ein Service, der die Zitate-Liste hält?

Konfiguration mit appsettings

Warum Konfiguration?

Gehört **nicht** in den Code:

- Connection Strings
- URLs anderer Dienste
- Limits und Feature-Schalter
- alles, was sich zwischen Development, Test und Production unterscheidet

> Code und Konfiguration trennen – derselbe Build läuft überall.

Konfigurationsquellen



- **Die spätere Quelle gewinnt** – pro Schlüssel, nicht pro Datei
- Umgebung: `ASPNETCORE_ENVIRONMENT` (Standard: `Production`)

Lesen mit IConfiguration

```
"Greeting": { "Text": "Willkommen im Kurs!", "Repeat": 2 }
```

```
public ConfigController(IConfiguration config)
{
    var text = config["Greeting:Text"];
    var repeat = config.GetValue<int>("Greeting:Repeat");
}
```

- Ebenen mit `:` trennen
- Als Umgebungsvariable: `Greeting__Text` (doppelter Unterstrich)

Das Options-Pattern

```
public class GreetingOptions
{
    public string Text { get; set; } = "";
    public int Repeat { get; set; } = 1;
}

// Program.cs
builder.Services.Configure<GreetingOptions>(
    builder.Configuration.GetSection("Greeting"));

// Controller
public ConfigController(IOptions<GreetingOptions> options)
    => _options = options.Value;
```

`IOptions` einmal beim Start · `IOptionsSnapshot` pro Request · `IOptionsMonitor` meldet Änderungen

Übung 6 – Werte aus appsettings lesen

15 Minuten

1. Abschnitt `Greeting` mit `Text` und `Repeat` in `appsettings.json`
2. `GreetingOptions` und `Configure<T>` in `Program.cs`
3. `GET /api/config` gibt den Text `Repeat`-mal aus
4. Text in `appsettings.Development.json` überschreiben

Bonus: Text per Umgebungsvariable `Greeting__Text` überschreiben

Lösung Übung 6

```
// appsettings.json
"Greeting": {
  "Text": "Willkommen im Kurs!",
  "Repeat": 2
}

// appsettings.Development.json
"Greeting": {
  "Text": "Hallo Entwickler!"
}
```

```
[HttpGet]
public string Get() ⇒
    string.Join(" ",
        Enumerable.Repeat(
            _options.Text,
            _options.Repeat));
```

Ergebnis in Development: „Hallo Entwickler!“ zweimal

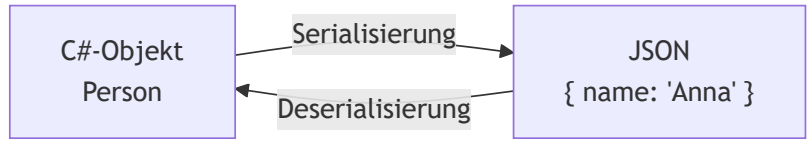
Exkurs: Geheimnisse gehören nicht in appsettings

- `appsettings.json` landet in Git
- Entwicklung: **User Secrets**
- Produktion: Umgebungsvariablen oder ein Tresor (z. B. Azure Key Vault)

```
dotnet user-secrets init
dotnet user-secrets set "Api:Key" "geheim123"
```

Serialisierung in JSON und XML

Was ist Serialisierung?



- ASP.NET macht beides automatisch
- Standard: JSON mit **System.Text.Json**

JSON-Konventionen in ASP.NET

Thema	Standardverhalten
Property-Namen	**camelCase:** `Name` → `name`
Einlesen	Groß-/Kleinschreibung egal
`null`-Werte	werden geschrieben
Enums	als Zahl
Einrückung	keine

> Ändern: pro Property mit Attributen, global mit `AddJsonOptions`

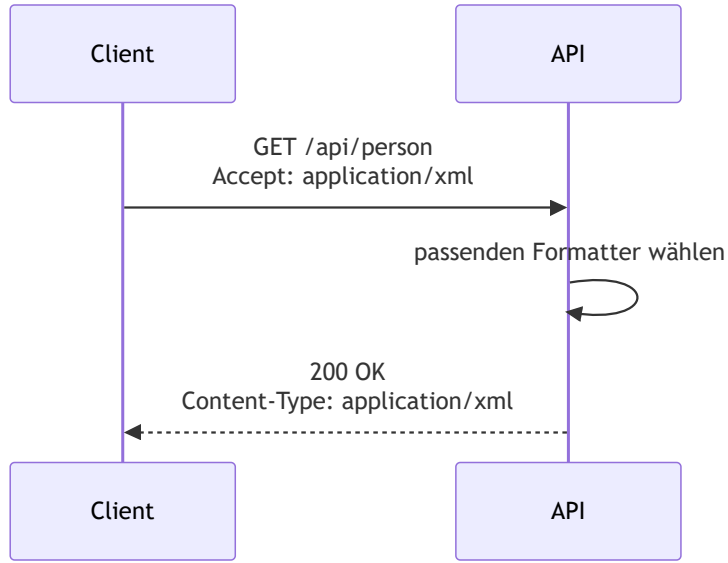
JSON anpassen

```
public class Person
{
    [JsonPropertyName("vollerName")]
    public string Name { get; set; } = "";

    [JsonIgnore]
    public string PasswordHash { get; set; } = "";
}

builder.Services.AddControllers().AddJsonOptions(o ⇒
{
    o.JsonSerializerOptions.WriteIndented = true;
    o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter());
});
```

Content Negotiation



- XML einschalten: `AddXmlSerializerFormatters()`
- Unbekanntes Format → standardmäßig trotzdem JSON

Übung 7 – JSON und XML

15 Minuten

1. Klasse `Person` mit `Name` und `Age`
2. `GET /api/person` gibt eine Person zurück
3. XML aktivieren
4. Einmal als JSON, einmal mit `Accept: application/xml` abrufen

Bonus: JSON eingerückt ausgeben & `[JsonPropertyName("alter")]` testen

Lösung Übung 7

```
builder.Services.AddControllers()
    .AddXmlSerializerFormatters()
    .AddJsonOptions(o ⇒ o.JsonSerializerOptions.WriteIndented = true);

[HttpGet]
public Person Get() ⇒ new Person { Name = "Anna", Age = 34 };
```

```
curl -k https://localhost:<port>/api/person
curl -k -H "Accept: application/xml" https://localhost:<port>/api/person
```

Datenmodelle

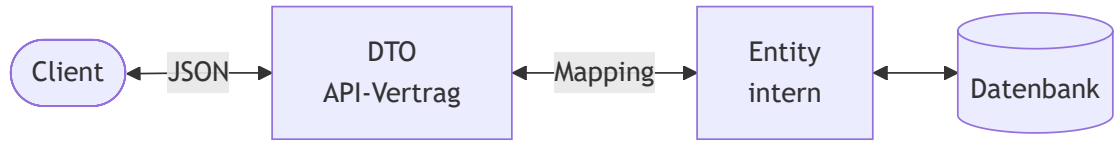
Was ist ein Modell?

- Klasse, die eine fachliche Sache beschreibt
- **POCO**: Plain Old CLR Object – Properties, keine Logik

```
public class Quote
{
    public int Id { get; set; }
    public string Text { get; set; } = "";
}

public record QuoteDto(int Id, string Text);   // kompakt, unveränderlich
```

Modell vs. DTO



- **Sicherheit** – interne Felder bleiben intern
- **Kein Overposting** – Client kann nicht `"isAdmin": true` setzen
- **Stabilität** – DB-Umbau ändert den API-Vertrag nicht

Typische DTOs pro Ressource

DTO	Richtung	Enthält
`CreateQuoteDto`	Client → API	Text, Autor – **ohne** Id
`UpdateQuoteDto`	Client → API	änderbare Felder
`QuoteDto`	API → Client	Id, Text, Autor

Mapping von Hand oder mit Bibliotheken wie **Mapster**, AutoMapper, …

Annotationen in Modellen

Attribut	Prüft
`[Required]`	Pflichtfeld
`[StringLength(50, MinimumLength = 2)]`	Länge
`[Range(0, 130)]`	Zahlenbereich
`[EmailAddress]`	E-Mail-Format
`[RegularExpression("…")]`	Muster
`[Url]`, `[Phone]`	URL, Telefonnummer

Namespace: `System.ComponentModel.DataAnnotations`

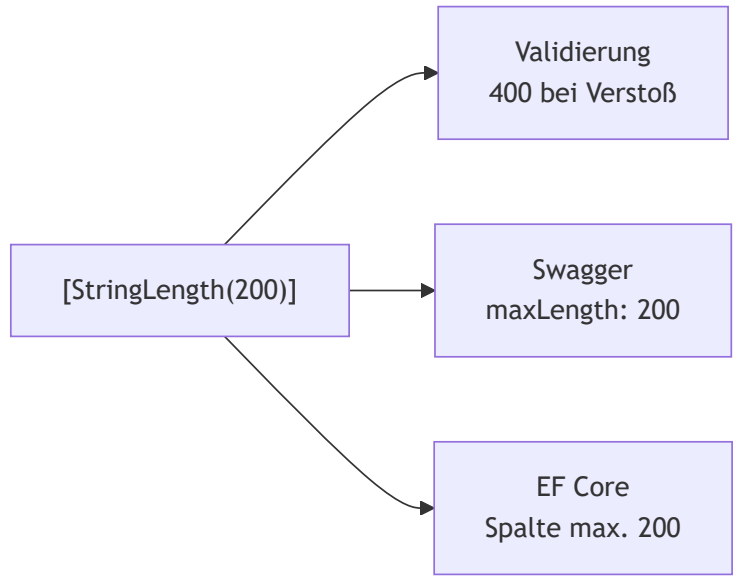
Annotationen im Code

```
public class PersonDto
{
    [Required(ErrorMessage = "Bitte einen Namen angeben")]
    [StringLength(50, MinimumLength = 2)]
    public string Name { get; set; } = "";

    [Range(0, 130)]
    public int Age { get; set; }

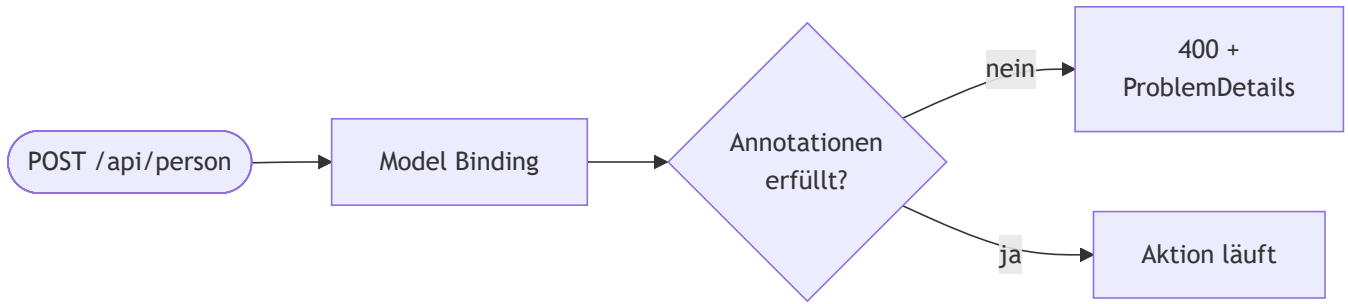
    [EmailAddress]
    public string? Email { get; set; }   // optional
}
```

Annotationen wirken an mehreren Stellen



Validation

Automatische Validierung



> Mit `[ApiController]` ohne eigenen Code – die Aktion sieht nur gültige Daten.

ProblemDetails

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Name": [ "The field Name must be a string with a minimum length of 2 ..." ],
    "Age":  [ "The field Age must be between 0 and 130." ]
  }
}
```

Standardformat für Fehler (RFC 9457) – Clients werten jeden Fehler gleich aus.

Eigene Validierung

```
public class EventDto : IValidatableObject
{
    public DateTime Start { get; set; }
    public DateTime End { get; set; }

    public IEnumerable<ValidationResult> Validate(ValidationContext ctx)
    {
        if (End ⩽ Start)
            yield return new ValidationResult(
                "Ende muss nach dem Start liegen", new[] { nameof(End) });
    }
}
```

Für komplexe Regeln: Bibliothek **FluentValidation**

Übung 8 – Annotationen und Validation

15 Minuten

`PersonDto` mit:

- Name: Pflicht, 2–50 Zeichen
- Alter: 0–130
- E-Mail: optional, gültiges Format

`POST /api/person` bauen, gültige und ungültige Daten senden

Testdaten: `{ "name": "A", "age": 200, "email": "keine-mail" }`

Bonus: eigene deutsche Fehlermeldungen

Lösung Übung 8

```
public class PersonDto
{
    [Required, StringLength(50, MinimumLength = 2)]
    public string Name { get; set; } = "";

    [Range(0, 130)]
    public int Age { get; set; }

    [EmailAddress]
    public string? Email { get; set; }
}

[HttpPost]
public IActionResult Create(PersonDto dto) ⇒ Ok(dto);
```

Versionierung

Warum versionieren?

Eine API ist ein **Vertrag**.

Bricht den Vertrag	Bricht ihn nicht
Feld umbenennen oder entfernen	neues optionales Feld
Datentyp ändern	neuer Endpoint
Feld zur Pflicht machen	zusätzlicher Query-Parameter

> Breaking Change → neue Version. Die alte läuft weiter, bis alle umgestellt haben.

Varianten der Versionierung

Variante	Beispiel
Pfad	`/api/v1/quotes`
Query	`/api/quotes?api-version=1.0`
Header	`X-Api-Version: 1.0`
Media Type	`Accept: application/json;v=1.0`

Im Kurs: Pfad – am sichtbarsten und am einfachsten zu testen

Asp.Versioning einrichten

```
dotnet add package Asp.Versioning.Mvc
dotnet add package Asp.Versioning.Mvc.ApiExplorer
```

```
builder.Services.AddApiVersioning(o ⇒
{
    o.DefaultApiVersion = new ApiVersion(1, 0);
    o.AssumeDefaultVersionWhenUnspecified = true;
    o.ReportApiVersions = true;
})
.AddMvc()
.AddApiExplorer(o ⇒
{
    o.GroupNameFormat = "'v'VVV";
    o.SubstituteApiVersionInUrl = true;
});
```

Versionierte Controller

```
[ApiController]
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/greeting")]
public class GreetingV1Controller
    : ControllerBase
{
    [HttpGet]
    public string Get()
        ⇒ "Hallo aus Version 1";
}
```

```
[ApiController]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/greeting")]
public class GreetingV2Controller
    : ControllerBase
{
    [HttpGet]
    public object Get() ⇒ new {
        Text = "Hallo aus Version 2",
        Zeit = DateTime.Now };
}
```

Übung 9 – Versionierung

15 Minuten

1. Pakete installieren, `AddApiVersioning` konfigurieren
2. `GET /api/v1/greeting` liefert einen Text
3. `GET /api/v2/greeting` liefert ein Objekt mit Text und Zeit
4. Header `api-supported-versions` mit `curl -k -i` prüfen

Bonus: beide Versionen in Swagger anzeigen (nächste Folie)

Swagger mit mehreren Versionen

```
builder.Services.AddSwaggerGen(o ⇒
{
    o.SwaggerDoc("v1", new() { Title = "KursApi", Version = "v1" });
    o.SwaggerDoc("v2", new() { Title = "KursApi", Version = "v2" });
});
// ...
app.UseSwaggerUI(o ⇒
{
    o.SwaggerEndpoint("/swagger/v1/swagger.json", "v1");
    o.SwaggerEndpoint("/swagger/v2/swagger.json", "v2");
});
```

Lösung Übung 9

- `/api/v1/greeting` → „Hallo aus Version 1“
- `/api/v2/greeting` → `{ "text": "…", "zeit": "…" }`
- Header: `api-supported-versions: 1.0, 2.0`
- `/api/greeting` ohne Version → 404

> Die Default-Version greift bei Query- oder Header-Versionierung, nicht bei festem Pfadsegment.

Challenge 2 – Zitate-API strukturieren

35 Minuten

Verwendet: alles von Tag 1, dazu DI, appsettings, Annotationen, Validation, Versionierung

> Reihenfolge: Service → Validierung → Konfiguration → Versionierung

Challenge 2 – Anforderungen

- Liste wandert in einen `IQuoteService` – **welche Lifetime?**
- Text: Pflicht, max. 200 Zeichen · Autor: optional, max. 100
- Maximale Anzahl aus appsettings, sonst `409 Conflict`
- v1 liefert Zitate **ohne** Autor, v2 **mit** Autor
- Alten `QuotesController` aus Challenge 1 löschen

Lösung Challenge 2 (1/4)

```
public class Quote
{
    public int Id { get; set; }

    [Required, StringLength(200)]
    public string Text { get; set; } = "";

    [StringLength(100)]
    public string? Author { get; set; }
}
```

```
public class QuoteOptions
{
    public int MaxQuotes { get; set; } = 10;
}
```

```
// appsettings.json
"Quotes": { "MaxQuotes": 5 }
```

Lösung Challenge 2 (2/4)

```
public interface IQuoteService
{
    IEnumerable<Quote> GetAll();
    Quote? Get(int id);
    Quote? Add(Quote quote);   // null = Maximum erreicht
    bool Delete(int id);
}
```

> Der Service kennt kein HTTP. Ob `null` zu 409 wird, entscheidet der Controller.

Lösung Challenge 2 (3/4)

```
public class QuoteService : IQuoteService
{
    private readonly List<Quote> _quotes = new();
    private readonly QuoteOptions _options;
    public QuoteService(IOptions<QuoteOptions> o) ⇒ _options = o.Value;

    public Quote? Add(Quote quote)
    {
        if (_quotes.Count ⩾ _options.MaxQuotes) return null;
        quote.Id = _quotes.Count == 0 ? 1 : _quotes.Max(q ⇒ q.Id) + 1;
        _quotes.Add(quote);
        return quote;
    }
    // GetAll, Get, Delete analog
}
```

```
builder.Services.AddSingleton<IQuoteService, QuoteService>(); // sonst leer
```

Lösung Challenge 2 (4/4)

```
[ApiController]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/quotes")]
public class QuotesV2Controller : ControllerBase
{
    // Konstruktor mit IQuoteService, GetAll, GetById wie bisher

    [HttpPost]
    public ActionResult<Quote> Create(Quote quote)
    {
        var created = _service.Add(quote);
        if (created is null) return Conflict("Maximale Anzahl Zitate erreicht.");
        return CreatedAtAction(nameof(GetById),
            new { id = created.Id, version = "2.0" }, created);
    }
}
```

v1: gleicher Aufbau, `[ApiVersion("1.0")]`, gibt nur `new { q.Id, q.Text }` zurück

Rückblick Tag 2

Rückblick Tag 2

- Services und DI mit Transient, Scoped, Singleton

Rückblick Tag 2

- Services und DI mit Transient, Scoped, Singleton
- appsettings, Umgebungen und das Options-Pattern

Rückblick Tag 2

- Services und DI mit Transient, Scoped, Singleton
- appsettings, Umgebungen und das Options-Pattern
- JSON, XML und Content Negotiation

Rückblick Tag 2

- Services und DI mit Transient, Scoped, Singleton
- appsettings, Umgebungen und das Options-Pattern
- JSON, XML und Content Negotiation
- Modelle, DTOs und Annotationen

Rückblick Tag 2

- Services und DI mit Transient, Scoped, Singleton
- appsettings, Umgebungen und das Options-Pattern
- JSON, XML und Content Negotiation
- Modelle, DTOs und Annotationen
- Automatische Validierung mit ProblemDetails

Rückblick Tag 2

- Services und DI mit Transient, Scoped, Singleton
- appsettings, Umgebungen und das Options-Pattern
- JSON, XML und Content Negotiation
- Modelle, DTOs und Annotationen
- Automatische Validierung mit ProblemDetails
- Versionierung

Ausblick Tag 3

Entity Framework Core · Migrationen · Logging · Tests

Die Lifetime-Frage bekommt morgen eine neue Antwort.



Mühlehner & Tavolato GmbH

SCHULUNG · BERATUNG · ENTWICKLUNG

AF

Alexander Fischer

af@mit.at

Kurse

offen und als Firmenschulung

Individualschulungen

zugeschnitten auf euer Team

Consulting

Beratung für eure Projekte

Auftragsentwicklung

wir bauen eure Software